# Phát Hiện Code AI hay Người Viết
# SemEval-2026 Task 13 Subtask A

**Lưu Thị Yến Nhi**
Khoa Khoa học Máy tính, VNU
25C11014

**Trần Thị Minh Anh**
Khoa Khoa học Máy tính, VNU
25C11029

**Abstract**

Báo cáo này trình bày hệ thống của chúng tôi tham gia SemEval-2026 Task 13 Subtask A – bài toán phân loại nhị phân nhằm phân biệt code do AI sinh ra và code do con người viết. Chúng tôi đề xuất hai kiến trúc độc lập: **HybridClassifier**, kết hợp embedding ngữ nghĩa từ Transformer với 11 đặc trưng phong cách (stylometric features) thông qua mạng Feature Gating; và **TLModel**, fine-tune mô hình DroidDetect-Base — vốn được pretrain cho phân loại malware Android — cho bài toán nhị phân. Thách thức cốt lõi là khả năng tổng quát hóa (*generalization*) sang ngôn ngữ lập trình và domain chưa thấy trong quá trình huấn luyện. Kết quả tốt nhất đạt được bởi TLModel (DroidDetect-Base, text-only) với **Accuracy = 0.6857** trên tập hidden test chính thức. Model này đạt F1-macro = 99.58% trên tập validation; lưu ý rằng cấu hình DroidDetect kết hợp agnostic features đạt F1-macro cao hơn (99.68%) nhưng cho kết quả test thấp hơn (0.5237), cho thấy agnostic features gây bias theo ngôn ngữ lập trình. Khoảng cách lớn giữa val và test phản ánh thách thức về domain shift và language shift đặc trưng của subtask này.

## 1 Giới Thiệu

Sự phát triển nhanh chóng của các mô hình ngôn ngữ lớn (LLM) đã tạo ra khả năng sinh code tự động với chất lượng ngày càng cao, đặt ra thách thức mới trong việc xác thực nguồn gốc của code [1]. SemEval-2026 Task 13 Subtask A yêu cầu xây dựng bộ phân loại nhị phân xác định liệu một đoạn code có được sinh bởi AI hay không, với thách thức chính là *khả năng tổng quát hóa* sang các ngôn ngữ lập trình và domain chưa xuất hiện trong tập huấn luyện.

Cụ thể, hệ thống được đánh giá trên bốn cài đặt:

- **Seen Languages & Seen Domains**: C++, Python, Java — Algorithmic
- **Unseen Languages & Seen Domains**: Go, PHP, C#, C, JavaScript — Algorithmic
- **Seen Languages & Unseen Domains**: C++, Python, Java — Research, Production
- **Unseen Languages & Unseen Domains**: Go, PHP, C#, C, JavaScript — Research, Production

Chúng tôi tiếp cận bài toán theo hai hướng song song: (1) khai thác đặc trưng phong cách viết code kết hợp với Transformer thông qua kiến trúc Hybrid, và (2) chuyển giao tri thức từ mô hình code pretrained (DroidDetect) thông qua Transfer Learning.

## 2 Dữ Liệu

### 2.1 Tập Dữ Liệu

Dữ liệu được thu thập từ cuộc thi SemEval-2026 Task 13 Subtask A, bao gồm các đoạn code từ nhiều ngôn ngữ lập trình khác nhau với nhãn nhị phân (0: AI-generated, 1: Human-written).

| Split | Số mẫu | Định dạng | Columns |
|---|---|---|---|
| Train | 500,000 | .parquet | code, generator, label, language |
| Validation | 100,000 | .parquet | code, generator, label, language |
| Test | 500,000 | .parquet | ID, code (không có nhãn) |

**Table 1.** Thống kê tập dữ liệu.

### 2.2 Tiền Xử Lý

Ngoài văn bản code thô, chúng tôi tính toán trước 11 đặc trưng phong cách (stylometric features) cho mỗi mẫu và lưu cache dưới dạng parquet để tái sử dụng, tránh tính lại trong mỗi lần thử nghiệm. Để tính perplexity, chúng tôi sử dụng Qwen/Qwen2.5-Coder-32B.

## 3 Phương Pháp

### 3.1 Kiến Trúc 1: HybridClassifier

HybridClassifier kết hợp hai luồng thông tin song song:

**Luồng ngữ nghĩa:** Đoạn code được mã hóa bởi một Transformer backbone (microsoft/deberta-v3-base hoặc microsoft/unixcoder-base). Thay vì dùng [CLS] pooling tĩnh, chúng tôi đề xuất **AttentionPooler** học trọng số động cho từng token:

$$s_i = \mathbf{w}^\top \tanh(\mathbf{W} h_i) \tag{1}$$

$$\alpha_i = \mathrm{softmax}(s_i) \tag{2}$$

$$\mathbf{v}_{\text{text}} = \sum_i \alpha_i h_i \tag{3}$$

trong đó $h_i$ là hidden state của token thứ $i$. Để xử lý padding, $s_i$ được masked fill bằng $-10^4$ tại các padding token trước khi qua softmax, đảm bảo $\alpha_i \approx 0$ tại những vị trí này.

**Luồng đặc trưng phong cách:** 11 đặc trưng thủ công được chiếu sang không gian 128 chiều qua **FeatureGatingNetwork**:

$$\mathbf{v}_{\text{feat}} = \mathrm{MLP}(\mathbf{f}) \in \mathbb{R}^{128}$$

với MLP gồm hai lớp Linear–BatchNorm–Mish–Dropout(0.2). Kích hoạt *Mish* [8] được chọn vì gradient mượt hơn ReLU, phù hợp với phân phối không đều của đặc trưng thủ công.

**Fusion:** Hai luồng được nối (concatenate) và đưa qua head phân loại nhị phân:

$$\hat{y} = \mathrm{Classifier}([\mathbf{v}_{\text{text}}; \mathbf{v}_{\text{feat}}])$$

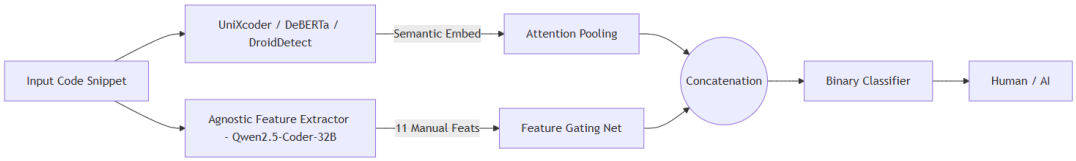


**Figure 1.** Kiến trúc HybridClassifier. Hai luồng text và stylometric được kết hợp trước khi phân loại.

## 3.2 Kiến Trúc 2: TLModel (Transfer Learning)

TLModel tận dụng mô hình project-droid/DroidDetect-Base — được pretrain cho bài toán phân loại malware Android dựa trên bytecode [2] — như điểm khởi đầu.

**Khởi tạo.** Chúng tôi load kiến trúc answerdotai/ModernBERT-base và khởi tạo encoder weights từ checkpoint DroidDetect:

1. Tải pytorch_model.bin từ DroidDetect Hub
2. Trích xuất keys với prefix text_encoder.*
3. Load vào ModernBERT với strict=False (bỏ qua head gốc)

**Forward pass.** Với chế độ text-only (use_agnostic_features=false):

$$\mathbf{p} = \mathrm{ReLU}(\mathbf{W}_{\mathrm{proj}} \cdot \bar{h})$$

trong đó $\bar{h}$ là mean pooling của các hidden states, và $\mathbf{p} \in \mathbb{R}^{128}$ là embedding sau projection.

### 3.3 11 Đặc Trưng Phong Cách

| # | Đặc trưng | Ý nghĩa |
|---|---|---|
| 1 | perplexity | PPL theo Qwen2.5-Coder; AI code predictable hơn |
| 2 | id_len_avg | Độ dài trung bình identifier |
| 3 | id_entropy | Shannon entropy của tên biến/hàm |
| 4 | id_short_ratio | Tỷ lệ identifier ngắn ≤2 ký tự |
| 5 | id_num_ratio | Tỷ lệ identifier có chứa chữ số |
| 6 | style_consistency | Mức độ trộn snake_case và camelCase |
| 7 | spacing_ratio | Tuân thủ spacing quanh toán tử |
| 8 | line_len_std | Độ lệch chuẩn độ dài dòng |
| 9 | ttr | Type-Token Ratio của từ vựng |
| 10 | comment_ratio | Tỷ lệ dòng comment |
| 11 | human_markers | Regex: TODO, FIXME, HACK, XXX |

**Table 2.** 11 stylometric features được trích xuất bởi AgnosticFeatureExtractor.

### 3.4 Hàm Mất Mát

**HybridClassifier.**

$$\mathcal{L} = \mathcal{L}_{\text{Focal}}(\gamma{=}2.0,\ \epsilon{=}0.1) + 0.05 \cdot \mathcal{L}_{\text{SupCon}}(\tau{=}0.3)$$

FocalLoss [7] giảm trọng số mẫu dễ, tập trung vào hard samples. SupConLoss [5] cluster hóa embeddings cùng nhãn để tạo decision boundary rõ hơn. Label smoothing $\epsilon = 0.1$ giảm overconfidence.

**TLModel.**

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{Triplet}}$$

với $\lambda = 0.1$. BatchHardMiner [4] chọn hard triplets trong mỗi batch, hiệu quả hơn random mining.

### 3.5 Chi Tiết Huấn Luyện

| Tham số | TLModel | HybridClassifier |
|---|---|---|
| Backbone LR | $1 \times 10^{-5}$ | $2 \times 10^{-6}$ |
| Head LR | $1 \times 10^{-4}$ | $1 \times 10^{-5}$ |
| Batch size | 128 | 128 |
| Gradient accum. | 2 (eff. 256) | 2 (eff. 256) |
| Epochs tối đa | 25 | 25 |
| Early stop patience | 5 | 5 |
| Max token length | 512 | 512 |
| Optimizer | AdamW ($\lambda_w = 0.01$) | |
| Scheduler | CosineAnnealingWarmRestarts | |
| Mixed precision | bfloat16 (RTX 5060) | |
| Gradient clip | $\|\nabla\|_2 \leq 1.0$ | |

**Table 3.** Hyperparameter cho hai kiến trúc mô hình.

Chúng tôi dùng **differential learning rates**: backbone được fine-tune với LR nhỏ hơn head 10 lần, nhằm tránh catastrophic forgetting [6].

**Augmentation**: Với sequence dài hơn max_length, thay vì cắt cố định từ đầu, chúng tôi chọn ngẫu nhiên một cửa sổ max_length token (*random window crop*) trong quá trình train. Val/test luôn dùng truncation từ đầu (deterministic).

## 4 Thực Nghiệm

Metric chính là **F1-macro** theo yêu cầu của ban tổ chức. Checkpoint tốt nhất được chọn dựa trên F1-macro trên tập validation.

### 4.1 Kết Quả Trên Tập Validation

| Model | Feat. | Epoch | Val Loss | Val Acc | Val F1 |
|---|---|---|---|---|---|
| DeBERTa-v3 | ✗ | 14 | 0.0569 | 98.43 | 98.42 |
| DeBERTa-v3 | ✓ | 4 | 0.2919 | 94.10 | 94.10 |
| UniXcoder | ✗ | 1 | 0.2095 | 96.18 | 96.18 |
| UniXcoder | ✓ | 10 | 0.0626 | 99.49 | 99.49 |
| DroidDetect | ✗ | 3 | 0.0135 | 99.58 | 99.58 |
| DroidDetect | ✓ | 3 | **0.0115** | **99.68** | **99.68** |

**Table 4.** Kết quả trên tập validation. **Feat.**: sử dụng agnostic features. ✓ = có, ✗ = không. Acc và F1 tính theo %.

### 4.2 Kết Quả Trên Hidden Test

| Model | Feat. | Test Acc |
|---|---|---|
| DeBERTa-v3 | ✗ | 0.4458 |
| DeBERTa-v3 | ✓ | 0.2888 |
| UniXcoder | ✗ | 0.2784 |
| UniXcoder | ✓ | 0.2214 |
| DroidDetect | ✓ | 0.5237 |
| **DroidDetect** | ✗ | **0.6857** |

**Table 5.** Kết quả trên tập hidden test chính thức. DroidDetect text-only đạt kết quả tốt nhất.

## 5 Phân Tích

### 5.1 Khoảng Cách Val–Test

Khoảng cách lớn giữa val F1 (≈99.6%) và test accuracy (0.69) là hiện tượng nổi bật nhất trong thực nghiệm này. Chúng tôi xác định ba nguyên nhân chính:

**1. Domain/Language shift.** Tập validation từ cùng phân phối với train (ngôn ngữ và domain tương tự). Tập hidden test bao gồm ngôn ngữ chưa thấy (Go, PHP, C#) và domain chưa thấy (Production, Research code). Model học các pattern ngôn ngữ-đặc thù thay vì pattern phân biệt Human/AI thực sự.

**2. Agnostic features bị bias.** Perplexity tính trên Qwen2.5-Coder phụ thuộc vào ngôn ngữ lập trình — cùng một đoạn code AI-generated viết bằng Go có perplexity khác hẳn Python. Mô hình dùng agnostic features đều cho kết quả test thấp hơn text-only, xác nhận hypothesis này.

**3. Overfitting vào val set.** Early stopping dựa trên val F1 đồng nghĩa với việc model được chọn là model overfit tốt nhất vào val distribution, không phải generalize tốt nhất.

### 5.2 Tại Sao DroidDetect Tốt Nhất?

DroidDetect được pretrain trên hàng triệu file APK bytecode — một domain code rất khác với code thuật toán thông thường. Điều này tạo ra encoder *ít bị bias* theo ngôn ngữ cụ thể hơn so với UniXcoder (pretrain trên GitHub) hay DeBERTa (pretrain trên text tự nhiên). Kết quả là embedding của DroidDetect tổng quát hóa tốt hơn khi gặp ngôn ngữ và domain chưa thấy.

### 5.3 Trực Quan Hóa Embedding

Hình 2 trình bày t-SNE [9] của embedding trên toàn bộ tập validation (100,000 mẫu) cho 3 backbone $\times$ 2 chế độ agnostic feature.

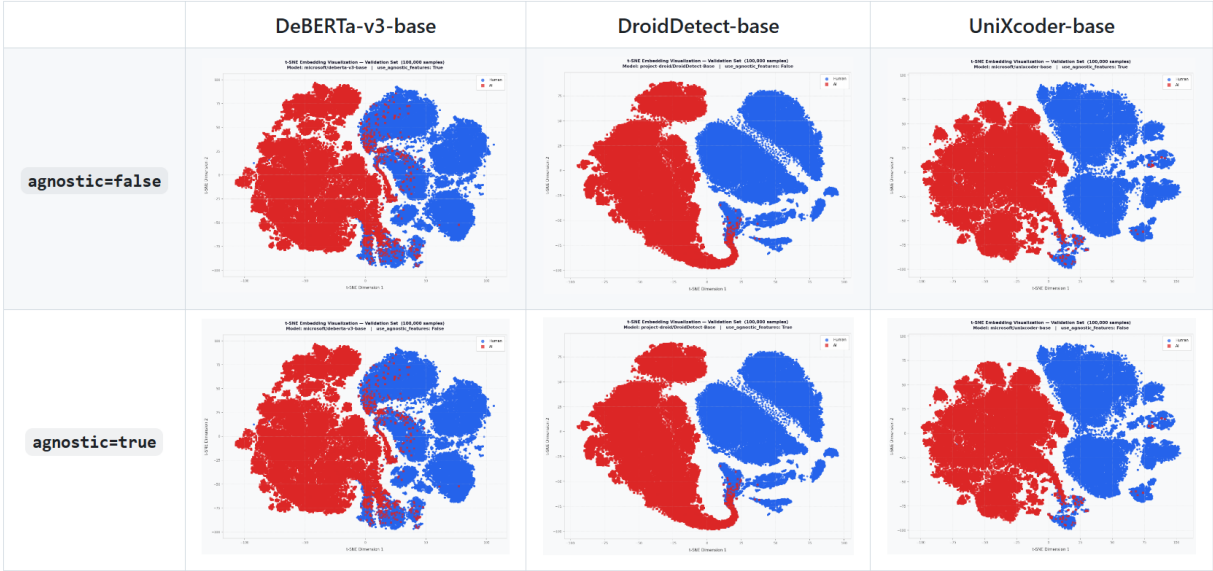


**Figure 2.** t-SNE embeddings trên tập validation (100,000 mẫu). Xanh: Human, Đỏ: AI-generated. Hàng trên: agnostic=false (text-only). Hàng dưới: agnostic=true (text + features).

Quan sát từ Hình 2:

**DroidDetect tạo boundary rõ nhất.** Cả hai chế độ của DroidDetect đều cho thấy hai cụm Human và AI phân tách gần như hoàn toàn, với vùng chồng lấp rất nhỏ ở biên giới. Điều này nhất quán với kết quả validation F1 = 99.58–99.68% và giải thích tại sao model này tổng quát hóa tốt hơn trên hidden test.

**DeBERTa và UniXcoder có vùng overlap lớn hơn.** DeBERTa (agnostic=false) cho thấy hai cụm lớn nhưng có các điểm bị xen lẫn đáng kể ở vùng trung gian — decision boundary kém sắc nét hơn. UniXcoder có pattern tương tự nhưng với cấu trúc cụm phức tạp hơn.

**Agnostic features ít thay đổi cấu trúc embedding.** So sánh hàng trên và hàng dưới, cấu trúc t-SNE không thay đổi đáng kể khi bật agnostic features — gợi ý rằng backbone Transformer vẫn là nguồn thông tin chủ đạo, agnostic features đóng vai trò phụ trợ.

## 5.4 Phân Tích Confusion Matrix So Sánh

Hình 3 trình bày confusion matrix của toàn bộ 6 cấu hình trên tập validation.

**Figure 3.** Confusion matrix trên tập validation (normalize theo hàng). Hàng trên: agnostic=false. Hàng dưới: agnostic=true.

| Model | Feat. | Human recall | AI recall |
|---|---|---|---|
| DeBERTa-v3 | ✗ | 0.98 | 0.98 |
| DeBERTa-v3 | ✓ | 0.97 | 0.91 |
| UniXcoder | ✗ | 1.00 | 0.99 |
| UniXcoder | ✓ | 0.98 | 0.94 |
| DroidDetect | ✗ | 1.00 | 1.00 |
| DroidDetect | ✓ | 1.00 | 1.00 |

**Table 6.** Recall theo từng class đọc từ confusion matrix. DroidDetect đạt recall hoàn hảo ở cả hai class trong cả hai chế độ.

Một số nhận xét quan trọng từ Hình 3 và Bảng 6:

**DroidDetect: hoàn hảo và ổn định.** DroidDetect đạt recall 1.00 cho cả Human và AI ở cả hai chế độ — mô hình không bị lệch về bất kỳ class nào và không bị ảnh hưởng bởi việc thêm agnostic features.

**Agnostic features làm giảm AI recall của DeBERTa và UniXcoder.** Khi bật agnostic features, AI recall của DeBERTa giảm từ 0.98 xuống **0.91** (giảm 7 điểm), và UniXcoder từ 0.99 xuống **0.94**. Hiện tượng này cho thấy agnostic features đưa thêm noise hoặc signal lệch vào quá trình phân loại — mô hình có xu hướng phân loại nhầm code AI thành Human hơn khi có thêm features. Đây là dấu hiệu của *feature interference*: thông tin từ stylometric features xung đột với thông tin ngữ nghĩa từ Transformer.

**Hàm ý với hidden test.** Mặc dù DroidDetect đạt recall hoàn hảo trên val, kết quả hidden test (0.69 vs ≈0.40–0.52 cho các model khác) cho thấy sự vượt trội này được duy trì một phần trên dữ

liệu unseen. Tuy nhiên, không có model nào đạt được mức recall cao tương tự trên hidden test, xác nhận rằng val set không đại diện đầy đủ cho distribution thực tế.

## 6 Các Cải Tiến Đề Xuất

**Cross-lingual augmentation.** Thêm dữ liệu code từ Go, PHP, C# vào tập huấn luyện để giảm language bias — hướng cải tiến có tác động lớn nhất.

**Perplexity chuẩn hóa theo ngôn ngữ.** Thay vì dùng perplexity tuyệt đối, chuẩn hóa bằng baseline perplexity của từng ngôn ngữ: ppl_norm = ppl/ppl_lang_mean.

**Ensemble.** Trung bình xác suất của DroidDetect và DeBERTa — hai backbone có inductive bias khác nhau thường bổ sung nhau.

**Domain-adversarial training.** Thêm gradient reversal layer với domain discriminator loss để encoder học feature ngôn ngữ/domain-agnostic [3].

## 7 Kết Luận

Chúng tôi đã trình bày hệ thống phân loại Human vs AI-generated code cho SemEval-2026 Task 13 Subtask A, với hai kiến trúc chính: HybridClassifier kết hợp Transformer và stylometric features, và TLModel fine-tune DroidDetect-Base cho bài toán nhị phân. Kết quả tốt nhất (**Test Acc = 0.6857**) đạt được bởi TLModel text-only, cho thấy rằng transfer learning từ mô hình code-specific mang lại khả năng tổng quát hóa tốt hơn so với các baseline Transformer thông thường.

Khoảng cách lớn giữa val và test (99.6% vs 68.6%) là bài học quan trọng về giới hạn của evaluation protocol khi val và train có cùng phân phối. Hướng cải tiến ưu tiên là augmentation đa ngôn ngữ và chuẩn hóa đặc trưng theo ngôn ngữ lập trình.

## References


[1] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Project Droid. Droiddetect-base: A pretrained model for android malware detection, 2024.

[3] Yaroslav Ganin et al. Domain-adversarial training of neural networks. 2016.

[4] Alexander Hermans, Lucas Beyer, and Bastian Leibe. In defense of the triplet loss for person re-identification. 2017.

[5] Prannay Khosla et al. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, 2020.

[6] James Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.

[7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of ICCV*, 2017.

[8] Diganta Misra. Mish: A self regularized non-monotonic neural activation function. *arXiv preprint arXiv:1908.08681*, 2019.

[9] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9, 2008.